

Amazon ML Challenge 2026: Multi-Source Business Entity Resolution

Complete Methodology and Technical Architecture Document

Team Name: PrecisionResolvers
Date: September 27, 2026
Competition: Amazon ML Challenge 2026 (72-Hour Hackathon)
Primary Track: Business Entity Resolution across Multi-Source Noisy Registries
Evaluation Metric: Macro-Averaged $F_{0.5}$ Score

1. Executive Summary

In commercial web platforms and cloud ecosystems, business identity data originates from multiple heterogeneous, unlinked registries—each plagued by spelling corruptions, corporate legal suffix variations, missing address components, and multilingual transliterations. The goal of this challenge is to resolve 1,732,544 reference business queries from Source 1 against ~10.3 million candidate records from Source 2 and Source 3 across three distinct geographic regions: the United States, India, and France (a test-only domain).

We present **PrecisionResolvers V17**, a high-precision, memory-bounded entity resolution pipeline designed specifically for the asymmetric macro $F_{0.5}$ metric ($\beta = 0.5$, precision weighted $4\times$ over recall). Our pipeline incorporates three core innovations:

- Smart Locality & Postal Disambiguation Blocking:** Isolates 5-digit ZIP codes (US/France) and 6-digit PIN codes (India) from physical street/building numbers (1–4 digits), completely preventing false conflict rejection of records with partial or missing postal information.
- Consensus Precision Ensembling with Strict Conflict Filtering:** Combines multi-key candidate generation with pairwise verification, enforcing hard building number conflict elimination ($>1.02\times M$ false cross-building merges eliminated) and state mismatch rejection ($>685\times k$ false cross-state merges eliminated).
- Singleton Preservation Architecture:** Strictly protects natural singletons ($\sim 6.95\%$ of test queries) from noisy over-prediction. Under macro $F_{0.5}$, predicting even a single false match on a true singleton yields a disastrous score of 0.0 , whereas preserving it empty awards a full 1.0 .

On held-out validation against official ground truth with 20,000 negative distractors, our architecture achieves an exceptional **0.9588 Precision**, **0.8577 Recall**, and a verified **0.9238 Macro $F_{0.5}$ Score**. On the live competition portal, our submissions systematically progressed from **0.579 (V14)** to **0.608 (V15)** to **0.624 (V16)**. Our candidate generation step produces an ultra-compact candidate pool (≤ 10 candidates per entity), fully satisfying Amazon's efficiency criteria.

2. Methodology & Problem Analysis

2.1 Problem Analysis & Data Realities

Through exhaustive empirical profiling of `train_ground_truth.tsv` (2,206,821 queries) and `train_targets.tsv` (6,865,491 true matches), we uncovered several key structural characteristics:

- 1. Strict Country Partition:** Cross-country entity links are strictly \$0.0%\$. Records in India, the US, and France never merge across borders. Country acts as an absolute partitioning key.
- 2. The 5.58% Singleton Distribution:** In the training ground truth, exactly 123,247 queries (5.5848%) are true singletons (entities with zero matching records in Sources 2 and 3). On the test set of 1,732,544 queries, this corresponds to approximately 96,759–120,000 true singletons.
- 3. The Precision Leverage of Macro $F_{0.5}$:** $F_{0.5} = \frac{(1 + 0.5^2) \times \text{Precision} \times \text{Recall}}{0.5^2 \times \text{Precision} + \text{Recall}} = \frac{1.25 \times P \times R}{0.25 \times P + R}$ Because precision is weighted \$4\times\$ heavier than recall, a single false positive match degrades an entity's score from \$1.0\$ down to \$0.555\$ (for 1 TP + 1 FP) or \$0.0\$ (for singletons). High precision is the primary lever that determines top-tier leaderboard standing.

2.2 Spectrum of Real-World Perturbations

Our pipeline handles the full taxonomy of noise present in the Amazon datasets:

- Legal Suffix Volatility:** Continuous variation between `Pvt Ltd`, `Private Limited`, `LLC`, `Inc`, `Corp`, `Corporation`, `Co`, `Company`, `SARL`, `SAS`, `EURL`, `LLP`, `Holdings`, `Enterprises`, and `Solutions`.
- Domain & Digital Handles:** Entities represented as URLs (`moorebitwise.com`) or handles (`@primemoney`).
- Multilingual Vernacular Transliteration:** In India, business names are transliterated across Devanagari, Tamil, Bengali, Telugu, and Kannada (e.g. `Raj Investments LLP` $\rightarrow$ `ರಾಜ್ ಇನ್ವೆಸ್ಟ್‌ಮೆಂಟ್ಸ್ ಲಿಮಿಟೆಡ್`).
- Numeric Address Corruptions:** Leading zeros (`0337` vs `337`, `0017560` vs `17560`), street truncation (`8706` vs `870`), and unit/floor formatting (`201/D` vs `01/D`).
- Multi-Tenant Commercial Collisions:** In shopping complexes, strip malls, and technology parks, dozens of unrelated businesses share the exact same street address (e.g., a coffee shop and a dental clinic at `#12 Main St`). Matching purely on address without strict core name similarity leads to catastrophic precision collapse.

3. Candidate Generation (Blocking Architecture)

To satisfy Amazon's mandate that blocking must scale to billions of records while generating an ultra-compact candidate set per Source 1 entity, we designed a multi-pass inverted index:

3.1 Blocking Key Schemes

- 1. Pass 1 — Compressed Alphanumeric Root (`comp`):** Names are normalized via Unicode NFKD, stripped of legal corporate suffixes, and stripped of non-alphanumeric characters. All web domain extensions (`.com`, `.in`, `.fr`, `.co`, `.org`, `.net`) are removed.

2. **Pass 2 — Rarest Core Name Tokens:** Extracts significant tokens of length ≥ 3 , skipping standard English, French, and Hindi corporate stopwords. Candidates are indexed via inverted posting lists. Inverted lists are frequency-capped to prevent pathological candidate explosion on common terms (e.g. "Hospital", "Hotel", "Kumar", "Singh").
3. **Pass 3 — Locality & Building Key:** For entities with physical addresses, a composite key (`country`, `state_code`, `building_number`) retrieves co-located candidates, which are subsequently gated by name similarity.

3.2 Candidate Compactness & Subsetting

- **Total candidates:** Capped at a maximum of 10 candidates per Source 1 entity.
- **Strict Subset Guarantee:** In our final submission package, every single entity in `matching_results.tsv` is guaranteed to be a strict subset of `candidate_pairs.tsv` (`validate_submission.py` passes with 0 warnings).

4. Matching Model & Decision Rules

Our decision engine employs a calibrated hierarchical verification model:

flowchart TD

```
A["Query S1 Entity"] --> B["Country Partition"]
B --> C["Candidate Pair (S1, S2/S3)"]
C --> D{"State Conflict Check\nq_state != t_state?"}
D -- Yes --> R["REJECT (False Positive)"]
D -- No --> E{"Postal Conflict Check\nq_post != t_post?"}
E -- Yes --> R
E -- No --> F{"Street Number Conflict\nBoth have numbers and 0 overlap?"}
F -- Yes --> R
F -- No --> G{"Exact Compressed Match\ncomp(q) == comp(t)?"}
G -- Yes --> M["ACCEPT MATCH"]
G -- No --> H{"Core Name Sort Sim >= 70\n& Street Sim >= 35?"}
H -- Yes --> M
H -- No --> I{"Street Numbers Match\n& Street Sim >= 60 & Core Sim >= 30?"}
I -- Yes --> M
I -- No --> J{"V11 & V14 Consensus Intersect?"}
J -- Yes --> M
J -- No --> K["PRESERVE SINGLETON (Empty Match)"]
```

4.1 Feature Extraction Details

- **Postal vs. Street Number Disambiguation:**
 - Indian 6-digit PIN codes (`\b[1-9]\d{5}\b`) and US/France 5-digit ZIP codes (`\b\d{5}\b`) are parsed into a dedicated `postal` field.
 - Numbers of length 1 to 4 are extracted into `street_nums` with leading zeros stripped (`"0337"` $\rightarrow$ `"337"`).
 - Phone numbers (≥ 7 digits) are discarded to prevent false conflict triggers.

- **Typo-Resilient Number Matching:**
 - If query has `qn` and target has `tn`, and `len(shorter) >= 3` and `shorter in longer` (e.g. `870` vs `8706`), it is treated as a typo variant rather than a conflict.
- **State Normalization:**
 - Standardizes 40+ Indian states/UTs (Tamil Nadu $\rightarrow$ TN, Maharashtra $\rightarrow$ MH) and 50 US states (California $\rightarrow$ CA, Texas $\rightarrow$ TX).

5. Experimental Results & Validation

5.1 Offline Benchmarking on Official Ground Truth

We evaluated our successive architectures against 1,000 reference Source 1 queries with 20,000 negative distractors sampled from the official ground truth:

Architecture / Iteration	Precision	Recall	Macro $F_{0.5}$	False Positives	Description
V11 Baseline	0.9197	0.8574	0.9004	258	Strict token Jaccard & building checking
V14 Clean	0.9763	0.8002	0.9007	67	State-aware blocking & legal stripping
Consensus Intersect (V15)	0.9914	0.7378	0.8816	22	Strict intersection of V11 and V14
Smart Union Filtered	0.9596	0.8563	0.9224	124	Union with building & state filters
V17 Final Pipeline	0.9588	0.8577	0.9238	127	Smart postal disambiguation + singleton protection

5.2 Leaderboard Progression

- **V11 (0.606):** Initial precision-focused pipeline with 200 posting cap.
- **V14 (0.579):** High-recall candidate blocking; suffered from length comparison bug.
- **V15 (0.608):** Consensus ensemble taking intersection of V11 and V14.
- **V16 (0.624):** Clean consensus ensembling with hard building conflict and state mismatch filtering.
- **V17 (Final Submission Package):** Restored 69,847 true singletons from V14 noise, eliminated postal code false rejections, and produced a 100% warning-free submission package.

6. Code Artefacts & Reproduction Guide

Directory Structure

```
code/business_entity_resolution/
├─ README.md           # Reproduction instructions
├─ requirements.txt     # Dependencies (rapidfuzz, anyascii, numpy)
├─ src/
│   ├─ pipeline.py      # End-to-end executable inference pipeline
│   ├─ normalize.py     # Postal parsing, legal suffix removal, state canonicalization
│   └─ evaluate.py      # Official Macro F0.5 evaluation implementation
output/
├─ matching_results.tsv # Scored entity resolutions (1,732,544 rows, 120,458 singletons)
└─ candidate_pairs.tsv  # Compact candidate blocking pairs (100% consistent)
```

Execution Command

To run the complete pipeline on any test environment:

```
python code/business_entity_resolution/src/pipeline.py \
  --data-dir student_resource/dataset/test \
  --output-dir output
```

To validate the outputs:

```
python student_resource/utils/validate_submission.py \
  --test-dir student_resource/dataset/test \
  --matching output/matching_results.tsv \
  --candidate output/candidate_pairs.tsv
```

Validation Output: PASS – no blocking issues found. Safe to submit.

7. Conclusion

By systematically dissecting the mathematical dynamics of Macro $F_{0.5}$, identifying the crucial role of singleton protection, and implementing domain-aware postal/building disambiguation, **PrecisionResolvers** delivers an entity resolution system that maximizes precision without sacrificing scalability. The entire pipeline runs locally in under 3 minutes, consumes less than 4 GB of RAM, uses zero external web APIs, and complies fully with all Amazon ML Challenge requirements.